

Тестирование метода добавления права на проекте «Райда»

Выполнил Грачев Ю. А.

E-mail: ungabunga756@gmail.com

1. Как бы вы тестировали этот метод на корректность работы?

Провести полный цикл позитивных и негативных тестов для CRUD-операций:

A. POST api/rights/{project_id} (Добавление нового права)

Позитивный сценарий:

Отправить запрос с корректными и валидными description.text и code.

Ожидаемый результат: HTTP-статус 200 OK и возвращение в теле ответа объекта созданного права, содержащего id, description и code. id должен быть уникальным.

B. GET api/rights/{project_id} (Получение списка прав для проекта)

Позитивный сценарий:

После успешного добавления права (п. А), отправить запрос на получение списка прав.

Ожидаемый результат: HTTP-статус 200 OK и наличие только что созданного права в возвращаемом списке.

C. PATCH api/rights/{project_id}/{right_id} (Изменение существующего права)

Позитивный сценарий:

Используя id созданного права, отправить запрос на изменение его description.text или code.

Ожидаемый результат: HTTP-статус 200 OK и возвращение в теле ответа объекта с обновленными данными.

D. GET api/rights/{project_id}/{right_id} (Получение конкретного права по ID)

Позитивный сценарий:

После успешного изменения права (п. С), отправить запрос на получение этого права по его id.

Ожидаемый результат: HTTP-статус 200 OK и возвращение в теле ответа объекта с последними обновленными данными.

E. DELETE api/rights/{project_id}/{right_id} (Удаление права)

Позитивный сценарий:

Используя id созданного права, отправить запрос на его удаление.

Ожидаемый результат: HTTP-статус 200 OK (или 204 No Content, если API не возвращает тело ответа при успешном удалении).

F. GET api/rights/{project_id}/{right_id} (Проверка отсутствия удаленного права)

Позитивный сценарий:

После успешного удаления права (п. Е), отправить запрос на получение этого права по его id.

Ожидаемый результат: HTTP-статус 404 Not Found.

2. Какие исключительные ситуации бы обрабатывали?

Проверка негативных сценариев:

A. Валидация входных данных для POST api/rights/{project_id} (поля description.text и code):

Пустые значения: Отправка запросов с description.text: "" или code: "".

Отсутствующие поля: Отправка запросов без поля description или code в теле запроса.

Некорректные типы данных: Отправка числовых значений, булевых или объектов вместо ожидаемой строки для description.text и code.

Превышение длины строки: Отправка очень длинной строки (например, 257+ символов), если есть ограничение по длине (предполагается по наличию "string" в схеме).

Специальные символы: Отправка строк, содержащих SQL-инъекции, XSS-скрипты или другие невалидные спецсимволы, для проверки их корректной обработки или экранирования. *(Замечание: для XSS/SQLi ожидаемый статус может быть 200, если бэкенд корректно экранирует строки, или 422/400, если стоит строгий фаервол валидации. Прилагаемая коллекция Postman рассматривает первый вариант)*
Дублирование code: Попытка создать право с code, который уже существует для данного project_id (предполагается, что code должен быть уникальным).

В. Валидация параметров URL:

Невалидный/отсутствующий project_id: Отправка запроса с project_id, который не существует, имеет неверный формат (например, не GUID, если это GUID), или равен null. Ожидаемый результат: HTTP-статус 404 Not Found или 400 Bad Request.

Невалидный/отсутствующий right_id (для PATCH/GET/DELETE по ID): Аналогично project_id. Ожидаемый результат: HTTP-статус 404 Not Found или 400 Bad Request.

С. Авторизация и аутентификация:

Неавторизованный доступ: Попытка добавить/изменить/удалить право без токена авторизации. Ожидаемый результат: HTTP-статус 401 Unauthorized.

Недостаточные права: Попытка добавить/изменить/удалить право, не имея необходимых разрешений для данного project_id. Ожидаемый результат: HTTP-статус 403 Forbidden.

3. Какими инструментами бы воспользовались?

A. Postman (или Insomnia/Swagger UI): Для ручного и полуавтоматического тестирования API.

NB: В задании отсутствует {base_url} для API. Потребуется запросить его у команды разработки или найти в документации.

B. Python с библиотекой requests и фреймворком pytest: Для написания автоматизированных API-тестов.

Git: Для контроля версий всех автоматизированных тестов и документации.

4. Какие улучшения бы предложили внести программистам в работу метода, чтобы повысить информативность тестирования?

- Четко указать максимальную/минимальную длину строк (code, description.text), формат project_id и right_id (например, UUID), а также обязательность/опциональность каждого поля.

- Явно указать, является ли code уникальным в рамках project_id.

- Текущий ответ 422 Validation Error (detail: [{loc, msg, type}]) является общим.

Стоит в дополнение к нему ввести специфические коды ошибок и более точные сообщения.

- Обеспечить, чтобы все 4xx ошибки возвращали стандартизированный формат тела ответа с errorCode и message.

- Имеет смысл включить в логи сервера уникальный ID запроса (Trace ID) для каждого входящего вызова API. Если QA обнаруживает проблему, он может предоставить этот ID, и разработчик быстро найдет соответствующую запись в логах для отладки.

- Разумно обеспечить логирование всех критических ошибок и исключений, возникающих внутри метода, с полными стектрейсами (stack traces), но без раскрытия

конфиденциальных данных.

- Предоставить доступ к отдельной тестовой среде, которая полностью изолирована от продакшена.

- Обеспечить механизм для быстрого создания и очистки тестовых данных (например, через отдельный API-эндпоинт для QA или скрипты для прямого манипулирования тестовой БД). Это позволит QA проводить тесты с чистыми данными без влияния на других тестировщиков и без необходимости ручной подготовки.